



GENERALITAT
VALENCIANA

Conselleria d'Educació,
Universitats i Ocupació

ceedcv

CENTRE ESPECÍFIC
D'EDUCACIÓ A DISTÀNCIA DE
LA COMUNITAT VALENCIANA

Sesión 03 — Análisis de requisitos e historias de usuario

2º CFGS DAW/DAM · Proyecto Intermodular II

Autores: Sergi García, Guillermo Garrido y Alfredo Oltra





La sesión en un mapa

De las **necesidades del cliente** al trabajo del sprint: convertir requisitos en **historias de usuario** valiosas, bien escritas, estimadas y priorizadas.



Fundamentos

¿Qué es un requisito?



Historias de usuario

Estructura y ejemplos



Calidad

Criterios, DoR y DoD



Estimación

Planning Poker y story points



INVEST

Buenas historias



Priorización

Valor vs. esfuerzo



Hilo conductor: seguimos con el caso **TaskFlow** — el mismo producto visto desde DAW, DAM y ASIR.



¿Qué es un requisito?

Un requisito es algo que el sistema debe hacer o una característica que debe tener.

El requisito...	Detalle
Define la frontera	Qué entra y qué queda fuera del producto
Guía el diseño	Base de la arquitectura y de las pruebas
Alimenta el backlog	Se convierte en historias de usuario
Se negocia	Se valida y se ajusta con el Product Owner



Requisito mal entendido = producto equivocado: es el error más caro de corregir, porque arrastra diseño, código y pruebas.



Tipos de requisitos



Funcionales — QUÉ hace

Funciones y comportamientos del sistema.

«El usuario puede registrarse con su email» ·
«El sistema notifica la asignación de una tarea».



No funcionales — CÓMO lo hace

Calidad: rendimiento, seguridad, usabilidad, disponibilidad.

«La lista de tareas carga en menos de 2 s» ·
«Los backups se conservan 7 días».



Los no funcionales también se escriben, negocian y prueban: «que vaya rápido» no es un requisito; «cargar en < 2 s» sí.



Historias de usuario: la idea

Explicación **informal y general** de una funcionalidad de software, contada desde la **perspectiva de quien la usa**.



Voz del usuario

Lenguaje natural,
sin tecnicismos



Centrada en valor

El porqué, no el
cómo técnico



Invita a conversar

El detalle se
acuerda con el
PO



Unidad del backlog

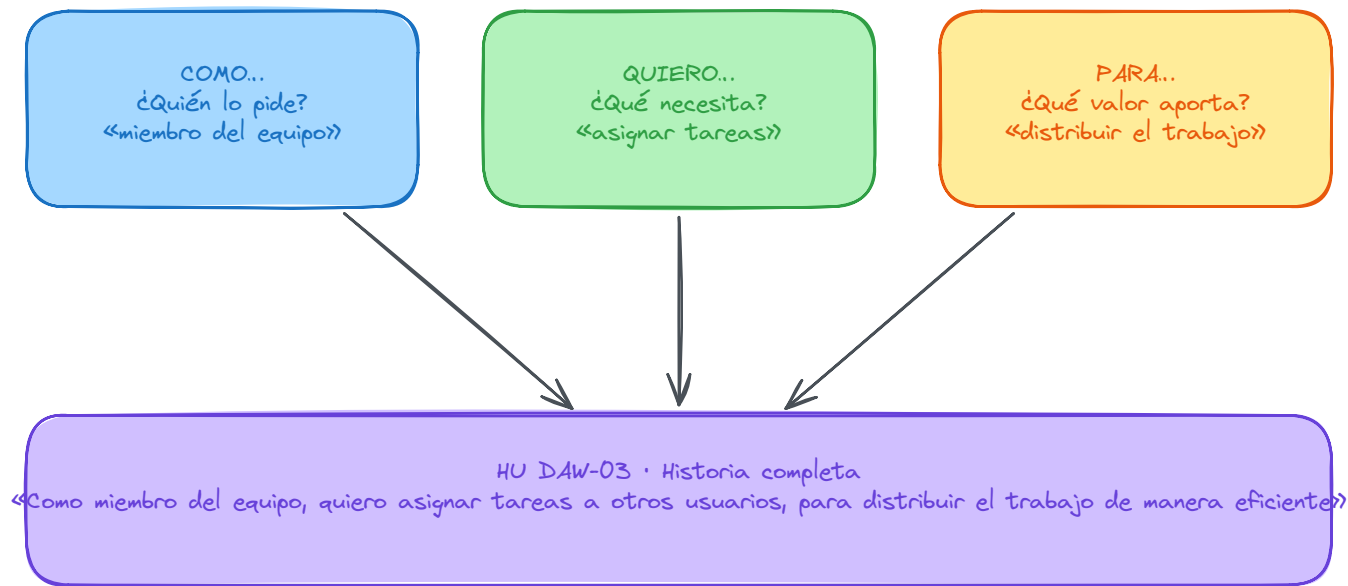
Se estima y se
planifica



La HU no sustituye a la especificación técnica: es el punto de partida de la conversación que



La estructura de una historia



Si no puedes rellenar el «para...», la historia no aporta valor claro: reescríbela o descártala.



Tres perfiles, un mismo producto

ID	Perfil	Historia de usuario
DAW-03	Web	Como miembro del equipo, quiero asignar tareas a otros usuarios para distribuir el trabajo.
DAM-02	Móvil	Como usuario móvil, quiero ver mis tareas ordenadas por prioridad para no olvidar las urgentes.
ASIR-03	DevOps	Como devops, quiero configurar backups automáticos de la BD para garantizar la recuperación ante fallos.

i TaskFlow es un solo producto, pero cada perfil (web, móvil, infraestructura) escribe sus historias desde su rol y su tecnología.



Criterios de aceptación

Condiciones **claras y medibles** que confirman que la historia está hecha como el PO esperaba.



HU-01: Como nuevo usuario, quiero registrarme con mi email y contraseña para poder acceder a mi cuenta.

Criterios de aceptación

- ✓ Los campos obligatorios se validan antes de crear la cuenta.
- ✓ Se comprueba el formato de email y contraseña; los errores se muestran con claridad.
- ✓ Tras el registro, el usuario accede a su cuenta automáticamente.



«Que funcione bien» no es un criterio: cada criterio debe poder verificarse con una prueba concreta.



Definition of Ready (DoR)

Condiciones que una historia debe cumplir **antes de entrar al sprint**.



Notificaciones: como usuario, quiero recibir notificaciones cuando se me asigne una tarea.

- 1 **Requisito validado con el PO** — el valor queda claro para todos.
- 2 **Contratos definidos** — endpoint `/api/notifications` acordado con los equipos.
- 3 **Diseño listo** — mockup del pop-up y comportamiento offline decididos.



Una historia que entra al sprint sin estar «ready» se convierte en bloqueos y sorpresas a mitad de semana.

Definition of Done (DoD)

La **barra de calidad** común que toda historia debe superar antes de darse por terminada.

DoR — antes del sprint

Puerta de **entrada**: requisito validado, diseño y contratos listos, sin bloqueos técnicos.

DoD — al cerrar la historia

Barra de **salida**: código compila, pruebas superadas, revisado por QA y aprobado por el PO.

 DoD del proyecto: el código compila sin errores y supera las pruebas instrumentadas — y además, revisión por pares y validación del Product Owner.



Estimación en equipo

Estimar es un **ejercicio colaborativo**: el debate descubre requisitos ocultos antes de comprometer el sprint.



Planning Poker

Cada persona revela su carta a la vez; las diferencias se discuten.



T-shirt sizes

XS · S · M · L · XL para estimaciones rápidas y gruesas.



Dot Voting

Votos repartidos para ordenar historias por complejidad o valor.



El resultado se expresa en **story points**: complejidad y esfuerzo relativos, nunca horas. En TaskFlow: asignar tareas = 5 · lista móvil = 3 · backups = 5.



INVEST: qué hace buena una historia

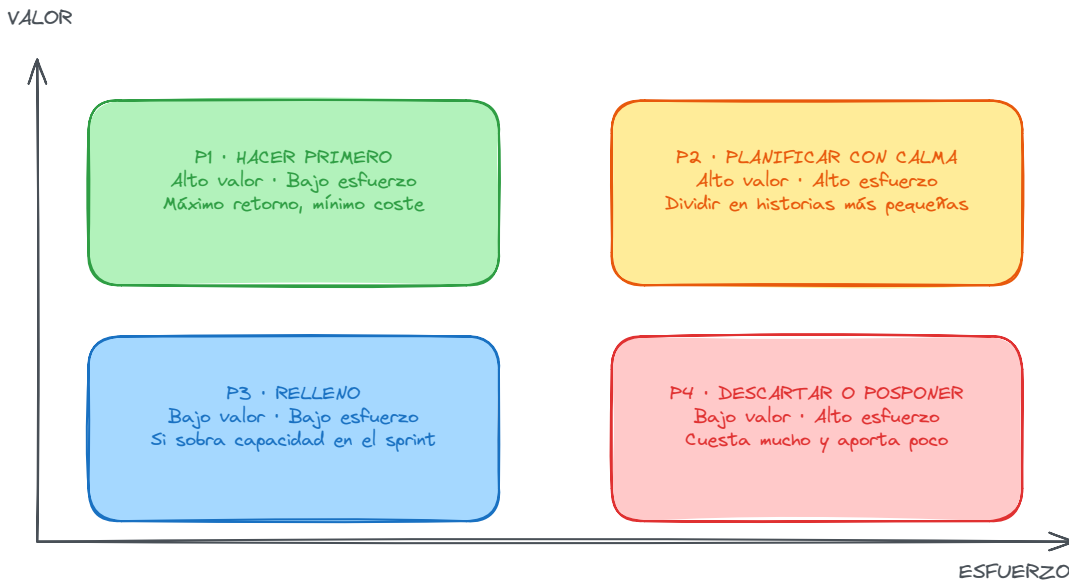
Letra	Criterio	Significado
I	Independent	Puede desarrollarse sin depender de otras
N	Negotiable	Se puede discutir y ajustar con el PO
V	Valuable	Aporta valor al usuario o al negocio
E	Estimable	El equipo puede estimar su esfuerzo
S	Small	Cabe en un solo sprint
T	Testable	Tiene criterios claros que permiten probarla



Usa INVEST al revisar el backlog: si una historia falla varias letras, **divídela o reescríbela**.



Priorizar: valor frente a esfuerzo



i El PO ordena el backlog con estos criterios: se implementa por **valor**, no por facilidad técnica.



Ejemplo completo — DAW-03: asignar tareas

Como *miembro del equipo*, quiero *asignar tareas a otros usuarios para distribuir el trabajo de manera eficiente*.

ID: DAW-03

5 story points · complejidad media

Prioridad: Alta (P1)

Depende de DAW-01 · DAW-02 · DAW-06

Criterios de aceptación

- Asignan solo usuarios con **permisos válidos**; si no, mensaje de error claro.
- **Confirmación visual** al asignar y notificación al asignado (panel y/o correo).
- Advertencia si el destinatario **ya no pertenece** al proyecto.
- Cambios visibles **al instante** para todo el equipo.

DoD y notas técnicas

- Tests unitarios y de integración **superados**; permisos validados.
- Campo `assigned_user_id` en el modelo de tareas.
- Endpoint REST de asignación integrado con notificaciones.
- Revisado por QA y **aprobado por el PO**.



Ejemplo completo — DAM-02: lista móvil priorizada

Como **usuario móvil**, quiero **ver mis tareas ordenadas por prioridad para no olvidar las más urgentes**.

ID: DAM-02

3 story points · baja-media

Prioridad: Alta (P1)

Depende de DAW-04 · DAM-01 · DAM-04

Criterios de aceptación

- Lista ordenada de **mayor a menor prioridad**.
- Cada tarea muestra **título, fecha límite, estado y color** de prioridad.
- Filtros por estado: pendiente, en progreso, completada.
- Sin tareas: «No tienes tareas pendientes. ¡Buen trabajo!»

DoD y notas técnicas

- Carga **menos de 2 s**, probado en Android e iOS.
- Endpoint `/api/tasks?order_by=priority desc`.
- RecyclerView / FlatList + **caché local**.
- Diseño según Figma (TaskFlow Mobile UI Kit).



Ejemplo completo — ASIR-03: backups automáticos

Como **DevOps**, quiero **configurar backups automáticos** de la base de datos para garantizar la **recuperación ante fallos**.

ID: ASIR-03

5 story points · complejidad media

Prioridad: Alta (P1)

Depende de ASIR-01 · ASIR-02

Criterios de aceptación

- Backup **diario y automático** a ubicación externa segura (S3, NAS...).
- Nombre con **timestamp**:
`taskflow_backup_20251015_2300.sql`.
- **Restauración** posible desde cualquier copia; retención ≥ 7 días.
- **Log** de cada copia y **alerta** si falla.

DoD y notas técnicas

- Cron o systemd timers + `pg_dump` / `mysqldump`.
- Subida segura con rclone o AWS CLI.
- Retención configurable:
`BACKUP_RETENTION_DAYS=7`.
- **Pruebas de restauración** periódicas.



Actividad: del requisito a la historia

Requisito de partida: «El sistema debe permitir enviar mensajes de texto a otros usuarios en tiempo real.»

- 1 Redacta tu HU** desde tu perfil — web (DAW), móvil (DAM) o infraestructura (ASIR) — con la plantilla «como... quiero... para...».
- 2 Define el DoR:** ¿qué debe estar listo antes del sprint (mockups, contratos de API, accesos)?
- 3 Escribe los criterios de aceptación:** claros, medibles y verificables.
- 4 Estimad y priorizad:** story points en equipo y posición en la matriz valor/esfuerzo.

✓ Entregable: ficha completa de la historia con el formato de los ejemplos — ID, historia, criterios, estimación, prioridad, dependencias y DoD.



Chequeo rápido de la sesión

1. ¿Qué diferencia hay entre un requisito funcional y uno no funcional?
2. Completa la plantilla: «Como ___, quiero ___ para ___». ¿Qué responde cada parte?
3. ¿Qué condiciones debe cumplir una historia antes de entrar al sprint (DoR)?
4. ¿En qué se diferencian los criterios de aceptación del DoD?
5. INVEST: ¿qué significa cada letra? ¿Cuál falla más a menudo en tus historias?
6. En la matriz valor/esfuerzo, ¿qué cuadrante se ataca primero y cuál se descarta?



Resumen de la sesión

- Los **requisitos** (funcionales y no funcionales) fijan qué debe hacer el sistema y cómo debe comportarse.
- Las **historias de usuario** los traducen a valor: «como *usuario*, quiero *acción* para *beneficio*».
- La calidad se concreta en **criterios de aceptación**, **DoR** (antes del sprint) y **DoD** (barra común de salida).
- **INVEST** filtra las buenas historias; la **estimación colaborativa** las mide en story points.
- La **matriz valor/esfuerzo** ordena el backlog: primero lo valioso y barato; fuera lo costoso e intrascendente.
- En **TaskFlow**, tres perfiles (DAW, DAM, ASIR) escriben historias distintas para el mismo producto.

